Course: BCA-602 (Sixth Semester)

System Software

BLOCK I – SYSTEM SOFTWARE & ASSEMBLER (Marks 30)

Q1. Define System Software and explain its classification.

Answer:
Introduction:
System software is a **set of programs that manage computer hardware and provide a platform for application software**. It acts as an intermediary between hardware and users.

Classification:

1. **Operating System (OS):** Manages resources, scheduling, file systems.
2. **Assemblers:** Convert assembly language to machine code.
3. **Compilers:** Translate high-level programming language to machine code.
4. **Linkers & Loaders:** Prepare executable programs.
5. **Utilities:** Perform system maintenance tasks (backup, antivirus, disk formatting).

Example:

- Windows OS, GCC Compiler, NASM Assembler.

Advantages:

1. Efficient hardware utilization.
2. Provides user-friendly interface for application programs.
3. Facilitates development of software applications.

Limitations:

1. Requires regular updates and maintenance.
2. Complexity may increase for large systems.

Q2. Explain the concept of an Assembler.

Answer:
Introduction:
An **assembler** is a system software that converts **assembly language programs into machine language instructions**. It allows programmers to write in **mnemonics** rather than binary code.

Explanation:

- Takes source code in assembly language.
- Translates instructions into machine code.
- Handles **symbols, literals, and labels**.

Example:

```
MOV AX, 5
ADD AX, 10
```

- Assembler converts these instructions into machine code.

Advantages:

- Simplifies low-level programming.
- Reduces human error compared to binary coding.

Limitations:

- Machine-specific: Each processor requires its own assembler.

Q3. Explain the Assembly Process with example.

Answer:
Introduction:
Assembly process involves converting **assembly language source code into executable machine code** in steps.

Steps:

1. **Lexical Analysis:** Break source code into tokens.
2. **Syntax Analysis:** Check instruction format and operands.
3. **Code Generation:** Convert instructions into binary machine code.
4. **Symbol Table Handling:** Map labels and variables to memory addresses.

Example:

```
START: MOV AX, 5
       ADD AX, 2
```

- START is stored in symbol table with address.
- Machine code generated for MOV and ADD instructions.

Advantages:

- Efficient code generation.
- Supports macros and reusable instructions.

Limitations:

- Debugging assembly code is difficult.

Q4. Explain Assembler Data Structures.

Answer:
Introduction:
Assemblers use **data structures** to manage symbols, literals, and instructions during assembly.

Important Data Structures:

1. **Symbol Table:** Stores variable/label names and their addresses.
2. **Literal Table:** Stores constants used in program.
3. **Opcode Table:** Stores machine code for each mnemonic.

Example:

- Label `LOOP` → Address `100`.
- Literal `5` → Memory location `200`.

Advantages:

- Efficient lookup and translation.
- Supports forward and backward referencing.

Limitations:

- Large programs require large tables, which increase memory usage.

Q5. Explain Macros and their use in Assembly Language.

Answer:
Introduction:
A macro is a **named sequence of instructions** that can be reused multiple times in a program. It simplifies repetitive coding.

Explanation:

- Defined once, called multiple times.
- Parameters can be used to customize macro behavior.

Example:

```
INCR MACRO REG
       ADD REG, 1
ENDM
```

- Using `INCR AX` increases AX by 1.

Advantages:

- Reduces code duplication.
- Increases programmer productivity.

Limitations:

- Excessive macro usage may increase program size.

Q6. Explain the role of Microprocessors in System Software.

Answer:
Introduction:
Microprocessors are **hardware CPUs** that execute instructions generated by system software. Assemblers, compilers, and OS interact directly with microprocessor architecture.

Example:

- Intel 8086 microprocessor executes machine code generated by assembler.

Advantages:

- Efficient execution of programs.
- Enables hardware-level optimization.

Limitations:

- System software must be compatible with processor architecture.

BLOCK II – LINKERS & LOADERS (Marks 20)

Q7. Explain the basic concepts of Linkers and Loaders.

Answer:
Introduction:
Linkers and loaders are system software that **prepare programs for execution** by combining modules and loading them into memory.

Explanation:

- **Linker:** Combines object files, resolves references, and creates executable.
- **Loader:** Loads executable into memory and prepares for execution.

Advantages:

- Supports modular programming.
- Resolves external references efficiently.

Limitations:

- Linking large programs may be time-consuming.

Q8. Explain Static and Dynamic Linking with examples.

Answer:
Introduction:
Linking connects **program modules** to form a single executable.

Static Linking:

- Combines all modules at compile-time.
- Example: C program with multiple `.obj` files linked into `.exe`.

Dynamic Linking:

- Modules linked at runtime.
- Example: Windows DLL files are dynamically linked.

Advantages:

- Static: Faster execution.
- Dynamic: Saves memory and allows module updates.

Limitations:

- Static: Cannot update modules without recompiling.
- Dynamic: Slightly slower at runtime.

Q9. Explain Loaders and their types.

Answer:
Introduction:
Loaders load executable programs into memory for execution.

Types:

1. **Absolute Loader:** Loads program at fixed memory location.
2. **Relocating Loader:** Adjusts addresses at load time.
3. **Direct Linking Loader:** Combines linking and loading dynamically.

Advantages:

- Enables execution of programs in memory.
- Supports modular and dynamic loading.

Limitations:

- Absolute loader lacks flexibility.

Q10. Explain Overlays in Loaders with example.

Answer:
Introduction:
Overlays allow **large programs to run in limited memory** by loading only needed parts.

Example:

- Graphics application: Only graphics module loaded when required.

Advantages:

- Efficient memory usage.
- Supports large programs on small memory systems.

Limitations:

- Requires careful program partitioning.

BLOCK III – COMPILERS (Marks 20)

Q11. Explain the introduction and importance of Compilers.

Answer:
Introduction:
A compiler is **system software that translates high-level language programs into machine code**. It enables faster execution and removes human errors in manual coding.

Advantages:

- Translates entire program at once.
- Generates efficient machine code.

Limitations:

- Compilation process can be slow.
- Debugging errors is indirect.

Q12. Explain phases of a Compiler in detail.

Answer:
Introduction:
Compiler translates source code to machine code in **multiple phases**.

Phases:

1. **Lexical Analysis:** Break source code into tokens.
2. **Syntax Analysis (Parsing):** Checks program structure against grammar rules.
3. **Semantic Analysis:** Checks for meaning, type consistency.
4. **Intermediate Code Generation:** Converts to intermediate representation.
5. **Code Optimization:** Improves performance.
6. **Code Generation:** Produces machine code.
7. **Symbol Table Management:** Tracks variables, functions, and addresses.

Example:

- C program: `int a=5;` is tokenized, parsed, and translated to machine instructions.

Advantages:

- Detects errors during compilation.
- Produces efficient executable code.

Limitations:

- Requires more memory.
- Complex compiler design.

Q13. Explain Lexical Analysis with example.

Answer:
Introduction:
Lexical analysis converts **source code into tokens** for further processing.

Explanation:

- Identifies keywords, identifiers, operators, constants.
- Removes comments and whitespace.

Example:
Source: `int x = 5;`
Tokens: `int` (keyword), `x` (identifier), `=` (operator), `5` (constant)

Advantages:

- Simplifies parsing phase.
- Early detection of lexical errors.

Limitations:

- Cannot detect syntax errors.

Q14. Explain Parsing in Compilers.

Answer:
Introduction:
Parsing checks **syntax correctness** using grammar rules.

Example:

- Expression: `a = b + 5;`
- Parser verifies `identifier = identifier + constant;`

Advantages:

- Ensures program structure is correct.
- Detects syntax errors before code generation.

Limitations:

- Complex grammar may require advanced parsing techniques.

Q15. Explain Intermediate Code Generation in Compilers.

Answer:
Introduction:
Intermediate code is a **machine-independent representation** of source code.

Example:

- Three-address code: `t1 = b + 5; a = t1;`

Advantages:

- Simplifies optimization.
- Makes compiler portable across architectures.

Limitations:

- Requires additional processing and memory.

Q16. Explain Symbol Table Management in Compilers.

Answer:
Introduction:
Symbol table tracks **identifiers, types, scope, and memory addresses**.

Example:

- Variable `int a` stored with type and memory location.

Advantages:

- Facilitates code generation and error checking.

Limitations:

- Large programs require large tables, consuming memory.

Q17. Explain Code Optimization in Compilers.

Answer:
Introduction:
Code optimization improves **speed, memory usage, or power efficiency** of generated code.

Example:

- Loop unrolling or removing redundant instructions.

Advantages:

- Faster execution.
- Reduces resource consumption.

Limitations:

- Can increase compilation time.

Q18. Explain Error Handling in Compilers.

Answer:
Introduction:
Compilers detect errors **during lexical, syntax, or semantic analysis**.

Techniques:

- Panic mode, phrase-level, and error recovery.

Example:

- Missing semicolon detected at parsing phase.

Advantages:

- Helps programmers correct errors before execution.

Limitations:

- Not all errors can be detected automatically.

Q19. Explain Macro Processors in Compilers and Assemblers.

Answer:
Introduction:
Macro processors allow **defining reusable code sequences** in programs.

Example:

```
INCR MACRO REG
  ADD REG, 1
ENDM
```

- `INCR AX` increments AX by 1.

Advantages:

- Reduces code redundancy.
- Simplifies programming.

Limitations:

- Excessive macros increase program size.

Q20. Explain Differences between Assembler, Linker, Loader, and Compiler.

Answer:

Feature	Assembler	Linker	Loader	Compiler
Function	Assembly → Machine Code	Combines object files	Loads program into memory	High-level → Machine Code
Stage	Source code translation	Post-compilation	Pre-execution	Compilation
Output	Object file	Executable	In-memory executable	Executable
Example	NASM	GCC linker	Windows Loader	GCC Compiler

Advantages:

- Each plays a unique role in program execution.

Limitations:

- Complexity increases when all stages are combined for large programs.